FlightLens: Literature Review

Project	FlightLens: US Flight Performance Data Platform
Program	Digital Egypt Pioneers Initiative (DEPI), Round 5
Track	AI & Data Science – Microsoft Data Engineer
Training provider	Skills Dynamix
Group	Group 3 · ALX5_AIS5_S6
Instructor	Ahmed Essam Azab
Team	Nepal Adel Ali, Asmaa Adel Ali, Omar Emad Hamdy, Abdelrahman Mahmoud Mohamed, Mosaab Mohamed Mohamed

---

1. Introduction

FlightLens turns raw US flight records from the Bureau of Transportation Statistics (BTS) into clean, tested, analysis-ready data, and serves it through a dashboard, a delay-prediction model and a plain-English question-answering assistant.

This review covers the work the project builds on, in six areas:

1. The cost of flight delays and the BTS on-time data
2. Flight delay prediction with machine learning
3. Data warehousing and dimensional modeling
4. Lakehouse architecture and big data processing
5. Pipeline orchestration and data quality
6. Natural-language questions over databases (text-to-SQL)

For each area, the review summarizes what earlier work found and explains how it shaped a FlightLens design decision. Section 8 describes the gap the project fills.

---

2. Flight delays and the BTS on-time data

2.1 Why delays matter

Flight delays are expensive for everyone involved. The *Total Delay Impact Study*, sponsored by the US Federal Aviation Administration (FAA) and carried out by the NEXTOR universities, estimated that domestic flight delays in 2007 cost the US economy **$32.9 billion**: $8.3 billion for airlines, $16.7 billion for passengers, $3.9 billion from lost demand, and a $4.0 billion reduction in GDP (Ball et al., 2010). The study also found that airlines add "padding" to their schedules to absorb expected delays, and that this hidden cost is as large as the cost of visible delays.

These figures explain why airline and airport managers need to know where, when and why delays happen, which are the business questions Q1–Q6 that FlightLens answers.

2.2 The BTS Airline On-Time Performance data

US carriers that meet the reporting threshold must report every scheduled domestic flight to the US Department of Transportation under federal rule 14 CFR Part 234. BTS publishes these records monthly in the Airline On-Time Performance database (Bureau of Transportation Statistics, n.d.). Each record contains the scheduled and actual departure and arrival times, delay minutes, cancellation and diversion flags, and, for flights that arrive 15 or more minutes late, the minutes attributed to five causes: carrier, weather, National Aviation System (NAS), security and late-arriving aircraft.

Because the data is complete and official, it is the most common source in academic flight-delay studies (Carvalho et al., 2021). However, it is published in a raw form: times are local `hhmm` numbers, cause columns are empty for most flights, and codes need lookup tables. Every analyst repeats the same cleaning. FlightLens addresses this by cleaning the data once and publishing it as a single trusted source.

---

3. Flight delay prediction

3.1 State of the research

Carvalho et al. (2021) published a systematic review of data science methods used in flight-delay research in *Transport Reviews*, with an earlier version on arXiv (Sternberg et al., 2017). The review classifies studies by **scope** (single airport, airline, or the whole network), **data** (flight records, weather, air-traffic data) and **method** (statistical models, operational research, probabilistic models, network models and machine learning). It reports that machine learning has become the dominant approach, and that the US BTS data is among the most used datasets.

Rebollo and Balakrishnan (2014) used Random Forests to forecast departure delays on the 100 most-delayed links of the US network, 2 to 24 hours ahead. Their features combined the delay state of individual airports with the delay state of the whole network. For a 2-hour forecast horizon, the average median test error was 21 minutes. Their work shows that recent delay history across the network is a strong predictor.

3.2 Choice of model

Gradient-boosted decision trees are a strong choice for tabular data with many categorical features such as carrier, airport and route. LightGBM (Ke et al., 2017) builds these trees with histogram-based splitting and leaf-wise growth, which makes it fast and memory-efficient on millions of rows. FlightLens trains a logistic regression baseline first, then LightGBM, so the gain from the more complex model is measured, not assumed.

3.3 Avoiding data leakage

A common mistake in delay prediction is to train on information that is only known after the flight, such as the actual departure time, taxi time or delay causes. Kaufman et al. (2012) define this problem as **leakage**: the model looks accurate in testing but fails in real use. FlightLens prevents leakage in two ways:

- only features known before departure are used (carrier, origin, destination, distance, scheduled hour, weekday, month, holiday flag, and historical on-time rates from earlier months);
- the data is split by time (train July–October, validate November, test December 2025), never randomly.

3.4 Choosing the right metric

Only about one flight in five arrives 15 or more minutes late, so the classes are imbalanced. Saito and Rehmsmeier (2015) showed that on imbalanced data the precision-recall curve is more informative than the ROC curve, because ROC can look good even when most positive predictions are wrong. FlightLens therefore reports both **ROC-AUC and PR-AUC**, and does not rely on accuracy.

---

4. Data warehousing and dimensional modeling

4.1 Two classic approaches

Inmon (2005) defines the data warehouse as a subject-oriented, integrated, time-variant and non-volatile collection of data, usually built as a normalized enterprise model first. Kimball and Ross (2013) propose **dimensional modeling**: business processes are stored as fact tables of measurements, surrounded by descriptive dimension tables. This **star schema** is easier for business users to understand and faster for analytical queries, because most questions become a simple join and group-by.

4.2 Application in FlightLens

FlightLens follows the Kimball approach, because its users are managers and analysts who filter and aggregate, not transactional systems. Following Kimball and Ross's four-step design process:

1. **Business process:** flight operations.
2. **Grain:** one row per scheduled flight in `fact_flights`. The finest grain was chosen so any question can be answered without re-loading data.
3. **Dimensions:** date, time (hour), carrier, airport and route. The airport dimension is used twice (origin and destination), which Kimball and Ross call a **role-playing dimension**.
4. **Facts:** delay minutes, delay flags, cancellation, diversion, taxi and air time, and the five delay causes.

An aggregate table (`agg_daily_airport_kpis`) stores daily results per airport, a technique Kimball and Ross recommend for fast dashboard pages.

---

5. Lakehouse architecture and big data processing

5.1 From data lakes to the lakehouse

Data warehouses give reliable, structured data but are costly and rigid; data lakes store any file cheaply but often become unreliable "data swamps." Armbrust et al. (2021) proposed the **lakehouse**, which keeps data in open file formats on cheap storage while adding warehouse features such as transactions and schemas. **Delta Lake** (Armbrust et al., 2020) implements this with a transaction log on top of Parquet files, giving ACID transactions, schema enforcement and "time travel" to earlier versions of a table.

5.2 The medallion architecture

Databricks describes the **medallion architecture**, which organizes a lakehouse into three layers of increasing quality: **Bronze** for raw data as received, **Silver** for cleaned and validated data, and **Gold** for business-level tables ready for reporting (Databricks, n.d.). Keeping the raw Bronze copy means any later layer can be rebuilt if a cleaning rule changes.

FlightLens uses exactly these three layers, with each step gated by quality checks. Its Gold layer is the star schema described in Section 4.

5.3 Distributed processing with Spark

Apache Spark processes large datasets in memory across many cores or machines, and offers SQL, streaming and machine-learning libraries in one engine (Zaharia et al., 2016). With 3,554,943 flights for six months and about 600,000 new rows each month, the data is too large to process comfortably in plain Pandas on a laptop, but well suited to PySpark. Because the same PySpark code runs on Azure Databricks, the project can move to the cloud without redesign.

---

6. Pipeline orchestration and data quality

6.1 Orchestration

A data pipeline has many steps that must run in order, on a schedule, and recover from failures. **Apache Airflow** defines pipelines as Python code in the form of directed acyclic graphs (DAGs), with scheduling, retries, backfills and monitoring built in (Apache Software Foundation, n.d.). FlightLens uses Airflow to download each new month, run the three layers, retry failed downloads three times and alert the team. Azure Data Factory provides the same role in the Microsoft cloud.

6.2 Data quality

Wang and Strong (1996) showed that data quality means more than accuracy: data consumers also judge data by completeness, consistency, timeliness and how easy it is to interpret. FlightLens turns these dimensions into automated checks with **Great Expectations**, an open-source framework that expresses data rules as testable "expectations" (Great Expectations, n.d.). Examples are: one row per flight, valid airport and carrier codes, cancelled flights with no actual arrival time, and every fact key present in its dimension. Rows that fail are moved to a quarantine area instead of reaching the Gold layer.

---

7. Plain-English questions over databases (text-to-SQL)

Text-to-SQL systems translate a natural-language question into a SQL query. The **Spider** benchmark (Yu et al., 2018) made this task harder and more realistic by testing models on complex queries over databases they had not seen during training. Large language models (LLMs) have since improved results sharply. Gao et al. (2024) compared prompt-engineering methods for LLM text-to-SQL and found that three choices matter most: how the database schema is represented in the prompt, which example questions are selected, and how the examples are organized. Their method, DAIL-SQL, reached 86.6% execution accuracy on Spider.

FlightLens applies these findings in its assistant: the prompt includes the Gold schema and the data dictionary, plus example questions with their SQL. Because generated SQL can be wrong, the assistant runs only `SELECT` statements through a read-only database user, shows the SQL to the user, and is measured against a 30-question test set with known answers.

---

8. Research gap and contribution

Area	What earlier work offers	What FlightLens adds
Delay data	Official, complete BTS records, but raw and hard to use	One cleaned, tested and documented source
Delay prediction	Many accurate models in research papers	A model trained only on trusted Gold data, with leakage checks, time-based validation and an API
Data modeling	Proven star-schema methods	A star schema designed for the questions airline and airport managers ask
Architecture	Lakehouse and medallion patterns, mostly on paid cloud platforms	The same pattern on free, open-source software in Docker, mapped one-to-one to Azure services
Access for non-technical users	Text-to-SQL research on benchmarks	A working assistant on real flight data, with safety rules and a measured accuracy

Most published flight-delay studies focus on the **prediction model** alone and assume clean data. Most data-engineering material describes architectures **in general**. FlightLens connects the two: it builds the full path from the raw government files to trusted data, and then shows that the dashboard, the model and the assistant all depend on that trusted data.

---

9. Conclusion

The literature supports each major FlightLens decision:

- **BTS data** because it is official and complete (14 CFR Part 234; Carvalho et al., 2021).
- **Medallion layers on Delta Lake and Spark** for reliable, re-runnable processing at scale (Armbrust et al., 2020, 2021; Zaharia et al., 2016).
- **A Kimball star schema** with one row per flight for fast, simple analysis (Kimball & Ross, 2013).
- **Automated quality checks** that cover more than accuracy (Wang & Strong, 1996).
- **LightGBM with leakage control and PR-AUC** for honest delay prediction (Ke et al., 2017; Kaufman et al., 2012; Saito & Rehmsmeier, 2015).
- **Schema-aware prompting with safety rules** for the text-to-SQL assistant (Yu et al., 2018; Gao et al., 2024).

References

Apache Software Foundation. (n.d.). *Apache Airflow documentation*. https://airflow.apache.org/docs/

Armbrust, M., et al. (2020). Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment, 13*(12), 3411–3424. https://doi.org/10.14778/3415478.3415560

Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*.

Ball, M., Barnhart, C., Dresner, M., et al. (2010). *Total delay impact study: A comprehensive assessment of the costs and impacts of flight delay in the United States*. NEXTOR, Federal Aviation Administration. https://rosap.ntl.bts.gov/view/dot/6234

Bureau of Transportation Statistics. (n.d.). *Airline On-Time Performance data*. US Department of Transportation. https://www.transtats.bts.gov/

Carvalho, L., Sternberg, A., Maia Gonçalves, L., Beatriz Cruz, A., Soares, J. A., Brandão, D., Carvalho, D., & Ogasawara, E. (2021). On the relevance of data science for flight delay research: A systematic review. *Transport Reviews, 41*(4), 499–528. https://doi.org/10.1080/01441647.2020.1861123

Databricks. (n.d.). *What is the medallion lakehouse architecture?* https://docs.databricks.com/en/lakehouse/medallion.html

Gao, D., Wang, H., et al. (2024). Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment, 17*(5), 1132–1145. https://doi.org/10.14778/3641204.3641221

Great Expectations. (n.d.). *Great Expectations documentation*. https://docs.greatexpectations.io/

Inmon, W. H. (2005). *Building the data warehouse* (4th ed.). Wiley.

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), Article 15. https://doi.org/10.1145/2382577.2382579

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146–3154).

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.

Rebollo, J. J., & Balakrishnan, H. (2014). Characterization and prediction of air traffic delays. *Transportation Research Part C: Emerging Technologies, 44*, 231–241. https://doi.org/10.1016/j.trc.2014.04.007

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Sternberg, A., Soares, J., Carvalho, D., & Ogasawara, E. (2017). *A review on flight delay prediction* (arXiv:1703.06118). arXiv. https://arxiv.org/abs/1703.06118

Wang, R. Y., & Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems, 12*(4), 5–33.

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3911–3921).

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM, 59*(11), 56–65. https://doi.org/10.1145/2934664